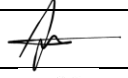


Assignment/Report Cover Sheet

Student Name	Student ID Number	Group No	Signature
ABDUL RAHIM BIN IDRIS	102368	GROUP 7	
PUTRA NURIMAN AKMAL BIN ROZIMAN	102917		
WAN ADAM DANISH BIN WAN ABIT	103014		
QHAIRUNNISA SHAHIRAH BINTI ZAKARIA	106089		

Subject Code : TMF2954	Subject Name : Java Programming
Assignment Title : Project Output (SDG 4: Quality Education)	Lecturer : Madam Azima Alya
Due Date : 5 th June 2026	Date Submitted : 5 th June 2026

This cover sheet must be completed, signed and firmly attached to the front of the submission. All work must be submitted by the due date. If an extension of work is granted, an assignment extension acknowledgement slip must be signed by the lecturer/tutor and attached to assignment. Please note that it is your responsibility to retain copies of your assignment.

c

Plagiarism and Collusion are methods of cheating that falls under Peraturan Akademik Universiti Malaysia Sarawak para 11: Etika Akademik

Plagiarism

Plagiarism is the presentation of work which has been copied in whole or in part from another person's work, or from any other source such as the internet, published books or periodicals without due acknowledgement given in the text.

Collusion

Collusion is the presentation of work that is the result in whole or in part of unauthorized collaboration with another person or persons.

Where there are reasonable grounds for believing that cheating has occurred, the only action that may be taken when plagiarism or collusion is detected is for the staff member not to mark the item of work and to report or refer the matter to the Dean. This may result in work being disallowed and given a fail grade or if the circumstances warrant, the matter may be referred to a Committee of inquiry for investigation. Such investigation may result in the matter being referred to the University Discipline Committee, **which** has the power to exclude a student.

MARK :



Table of Contents

1.0 Explanation of classes and interfaces	2
2.0 Application Interfaces and Explanation.....	3
3.0 Testing and Debugging	4
4.0 Proof of Discussions	5

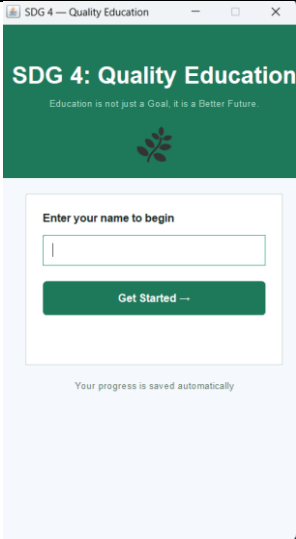
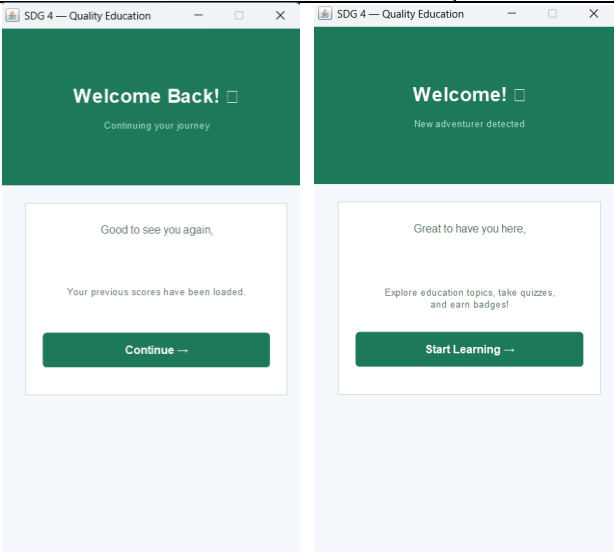
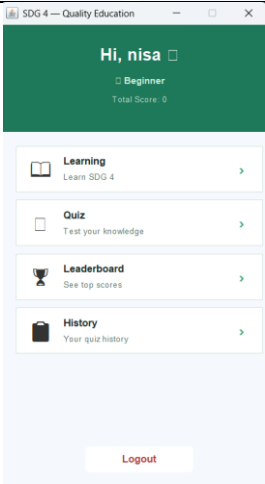
1.0 Explanation of classes and interfaces

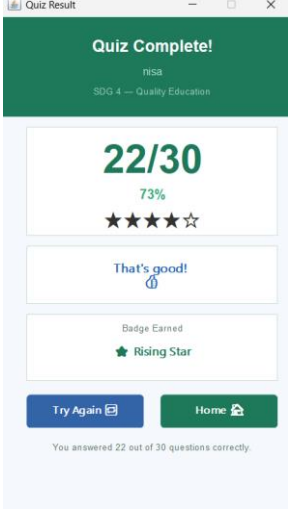
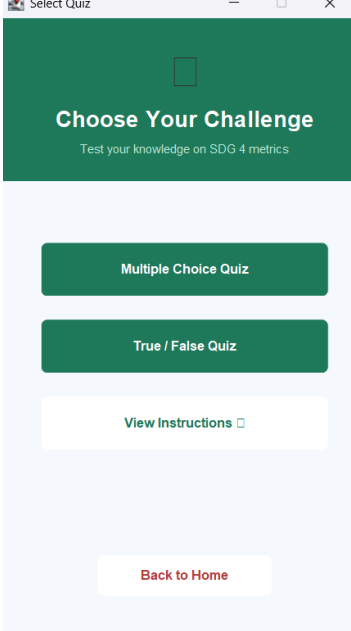
Name	Description	Role in Implementation	Assigned member
Home	Home is the main navigation controller of the entire application. It is the first screen the user sees and manages all panel switching through Java CardLayout.	Acts as the application entry point and owns LearningModule and MenuQuiz via composition. Uses LeaderBoard and HistoryPage via dependency. Throws InvalidInputException when the user enters a blank name. Holds user typed as IUser and demonstrates programming to an interface	Qhairunnisa Shahirah
User	User is the data model class. It stores the current player's name and score and handles all reading from and writing to users.txt.	Implements all IUser methods with @Override, setScore() is overloaded demonstrates Method Overloading. saveUser() and loadUser() fulfill the Data Storage requirement and getBadge() drives the gamification badge display in Home	Qhairunnisa Shahirah
QuizResultPage	Displayed after a quiz ends. It shows the player's score, percentage, star rating, motivational message and badge earned.	Implements IResultDisplay all 5 methods use @Override. SaveResult uses meaningful exception handling. Stars and badges are gamification elements. Called by MCQModule and TrueFalseModule after quiz ends	Qhairunnisa Shahirah
IUser	Contains user data operations where 11 method covering name input, score management, file operations and navigation.	Acts as the bridge between Home and User. Instead of declaring user as type User, Home declares it as a type IUser. The interface also ensures key methods like getBadge() are always available for the gamification badge display, and getLearningModule(), getQuizMenu(), getHistory() serve as navigation point connecting the user data layer to the app's screens.	Qhairunnisa Shahirah
IResultDisplay	IResultDisplay is the contract for the result screen. It ensures QuizResultPage	IResultDisplay contains five method signatures, showResult() to display the final score,	Qhairunnisa Shahirah

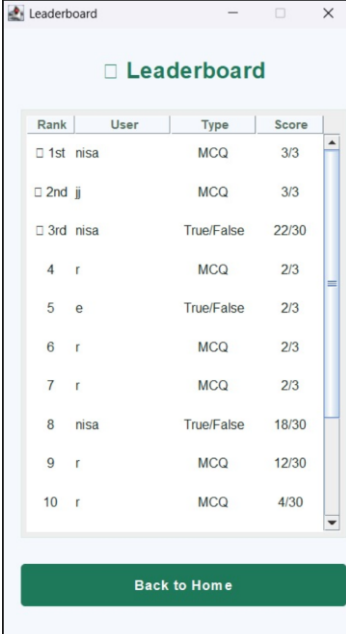
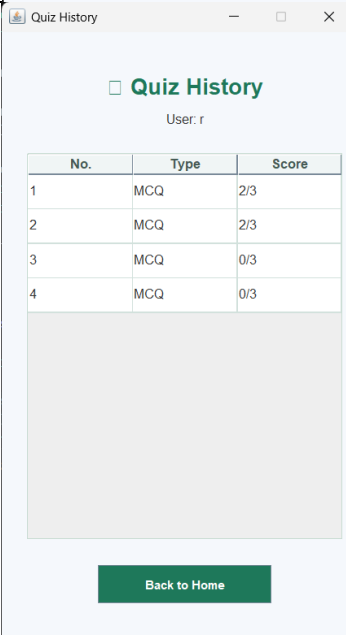
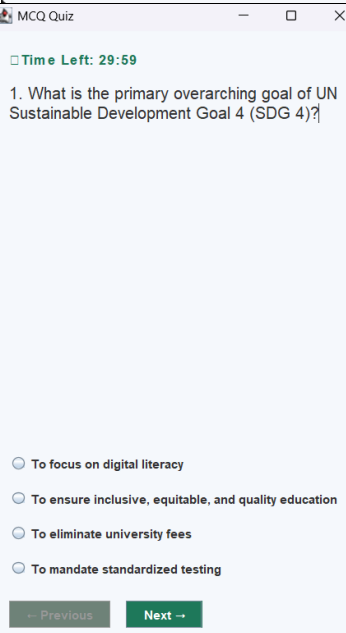
	always provides five key behaviours.	showMotivationalMessage() for the feedback, getScorePercent() to calculate the percentage, saveResult() to write the score to file, and showHome() to navigate back	
AbstractQuizModule	Abstract parent class managing shared quiz logic, state data variables and common file system operations.	Establishes reusable hooks for runtime metrics such as stopwatch countdown, startTimer() and handles append-mode operations to persists scores to QuizScores.txt	Abdul Rahim and Wan Adam
IQuiz	Java interface establishing strict behavioural method signatures that controls all core quiz operations.	Formulates system constraints by commanding specific operations (loadQuestion, checkAnswer, showResult) that implementing concrete subclasses must override.	Abdul Rahim
MCQModule	Concrete subclass that extends AbstractQuizModule to handle multiple-choice interactive test configurations.	Instantiate a 30 questions SDG 4 dataset, render individual choices using a vertical BoxLayout radio group, and tracks individual selection updates.	Abdul Rahim
TrueFalseModule	Concrete subclass that extends AbstractQuizModule to handle binary evaluation test tracks.	Instantiates a 30 questions SDG 4 boolean dataset, manages explicit validation listeners for custom True/False toggle states, and displays a live visual clock.	Abdul Rahim
ILearning	Java interface defining the structural contract for the educational content module.	Establishing the required navigation and rendering methods (createPage, showScreen, nextSlide, prevSlide) that any learning module must implement.	Wan Adam Danish
LearningModule	Concrete subclass that implements ILearning to display SDG 4 educational content	Utilizes a CardLayout to manage 10 distinct pages of content. It dynamically scales and renders .jpeg poster images proportionally and handles interactive GUI navigation to progress through slides or return to the dashboard.	Wan Adam Danish
IMenuQuiz	Java interface establishing the behavioral contract for the quiz selection	Defines the required methods for rendering the selection UI (showMenuQuiz) and handling the routing logic (selectQuizType)	Wan Adam Danish
MenuQuiz	Concrete subclass the implements	Provides a GUI for the user to choose between MCQ	Wan Adam Danish

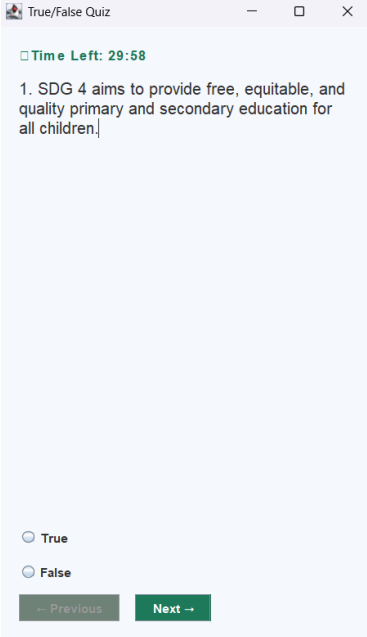
	IMenuQuiz to act as navigational bridge between the dashboard and the quizzes	and True/False formats. Crucially, it acts as an integration bridge by accepting userName and parentHome references and passing them into the chosen quiz module to ensure data persistence and reverse navigation	
ILeaderBoard	Java interface defining the structural contract for the leaderboard module.	Establishes the required methods for leaderboard functionality including displayLeaderboard(), clearLeaderboard(), showHome(), and getTopScore() to ensure consistent leaderboard behaviour.	Putra Nuriman Akmal
LeaderBoard	Concrete class that implements ILeaderBoard to manage and display quiz ranking	Reads quiz score records from QuizScores.txt, sorts scores based on performance, displays leaderboard rankings, and provides navigation back to the Home screen.	Putra Nuriman Akmal and Abdul Rahim
IHistoryDisplay	Java interface defining the structural contract for the quiz history module.	Establishes the required methods for displaying user history and navigation, ensuring all history-related implementations follow a consistent structure.	Putra Nuriman Akmal
HistoryPage	Concrete class that implements IHistoryDisplay to display user quiz history records.	Reads quiz attempt records from QuizScores.txt, filters records based on the current user, displays previous quiz attempts, and provides navigation back to the Home screen.	Putra Nuriman Akmal

2.0 Application Interfaces and Explanation

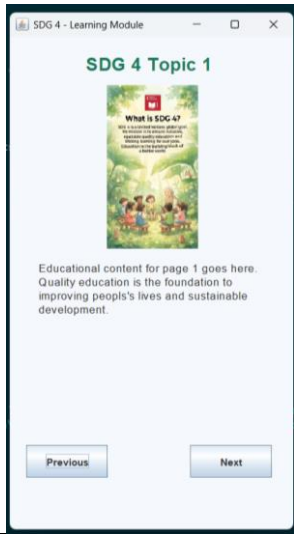
Interfaces	Explanation
 The screenshot shows the login page for 'SDG 4: Quality Education'. It features a green header with the title and a subtitle 'Education is not just a Goal, it is a Better Future'. Below the header is a white box with the prompt 'Enter your name to begin', a text input field, and a green 'Get Started -->' button. At the bottom, it says 'Your progress is saved automatically'.	The Login page, which utilizes the IUser interface, prompts the user to enter their name. It then verifies whether they are a new or returning user before saving their login details to a users.txt file.
 Two side-by-side screenshots of the welcome page. The left one says 'Welcome Back!' with 'Continuing your journey' and 'Good to see you again, Your previous scores have been loaded.' and a 'Continue -->' button. The right one says 'Welcome!' with 'New adventurer detected' and 'Great to have you here, Explore education topics, take quizzes, and earn badges!' and a 'Start Learning -->' button.	This welcome page, implemented as part of the IUser interface, greets users after they enter their name. The system checks the user's login history to display a customized message: showing "Welcome Back!" for returning users, and "Welcome! New adventurer detected" for those accessing the application for the first time.
 The screenshot shows the main menu for 'Hi, nisa'. It displays 'Beginner' and 'Total Score: 0'. There are four menu items: 'Learning' (Learn SDG 4), 'Quiz' (Test your knowledge), 'Leaderboard' (See top scores), and 'History' (Your quiz history). Each item has a right arrow. At the bottom is a red 'Logout' button.	The SDG 4-Quality Education features four navigation options. User can choose to learn, try to do the quiz, view the leaderboard, and look for their history performance. Total score increase based on the score previously if the user used to used the system.

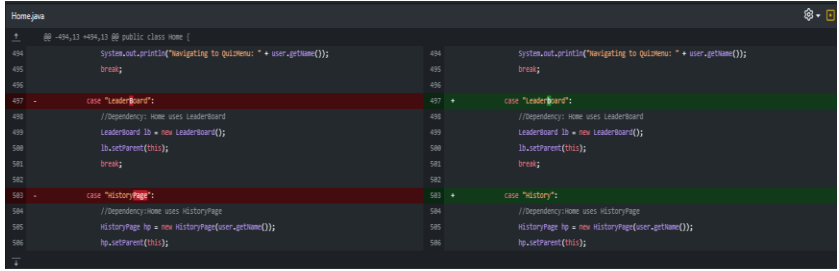
 The Quiz Result interface shows a green header with 'Quiz Complete!' and 'nisa SDG 4 - Quality Education'. It displays a score of 22/30 (73%) with five stars (four filled). A message 'That's good!' is shown. Below, it says 'Badge Earned' and shows a 'Rising Star' badge. At the bottom are 'Try Again' and 'Home' buttons. A footer note says 'You answered 22 out of 30 questions correctly.'	This Quiz Result interface features star ratings and achievement badges (such as the 'Rising Star' badge) to implement the required gamification elements. It calculates the user's total marks, converts them into a percentage, and displays a dynamic motivational message like 'That's good!'. At the bottom, 'Try Again' and 'Home' buttons are included, allowing users to easily retake the quiz or return to the main menu to explore other options.
 The Menu Quiz interface has a green header with 'Choose Your Challenge' and 'Test your knowledge on SDG 4 metrics'. It features two green buttons: 'Multiple Choice Quiz' and 'True / False Quiz'. Below these is a 'View Instructions' button with a right arrow. At the bottom is a 'Back to Home' button.	This is the Menu Quiz interface establishes the behavioral contract for the quiz selection, ensuring standard routing methods are present. The MenuQuiz class implements this interface to serve as the application's central routing hub. It allows users to choose between Multiple Choice and True/False formats.
 The Learning Module interface is titled 'SDG 4 Topic 1'. It features an illustration of children in a forest. Text reads: 'What is SDG 4? SDG 4 is a United Nations global goal. Its mission is to ensure inclusive, equitable quality education and lifelong learning for everyone. Education is the building block of a better world.' Below the illustration, it says 'Educational content for page 1 goes here. Quality education is the foundation to improving people's lives and sustainable development.' At the bottom are 'Previous' and 'Next' buttons.	The Learning Module serves as the primary educational component of the application. It's objective is to provide users information regarding Sustainable Development Goal (SDG) 4: Quality Education before they attempt the quizzes. The interface is built using Java Swing and relies heavily on CardLayout manager allowing the application to stack 10 distinct 10 UI panels (cards) on top of each other, displaying only one at the time.

	The LeaderBoard interface is responsible for displaying the ranking of users based on their quiz performance. The system reads quiz records stored in QuizScores.txt, calculates the score percentage for each attempt, and sorts the results from highest to lowest. This allows users to compare their achievements with other participants. A "Back to Home" button is also provided to enable users to return to the main navigation menu.
	The HistoryPage interface allows users to review their previous quiz attempts and performance records. The system retrieves user-specific data from QuizScores.txt and displays all quiz attempts associated with the current user. This feature helps users monitor their learning progress and identify areas for improvement. A navigation button is provided to allow users to return to the Home screen after viewing their history.
	The MCQModule provides users with the ability to answer questions in the form of multiple-choice questions. Options are given in the form of radio buttons that lie under JRadioButton. The module also includes the timer for users to keep track of their time while answering questions. In addition, the “Previous” and “Next” buttons are included to navigate users to previous or next questions.

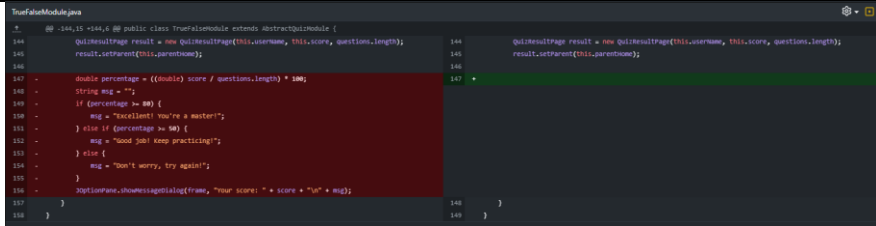
	The TrueFalseModule is a module that is used for users to answer true or false questions by using logical boolean value. The timer is included to keep track of students while answering the questions. Furthermore, JRadioButton allows users to pick one of the True or False radio options. Buttons such as “Previous” and “Next” are located under the TF options to navigate users to the next or previous questions.
---	---

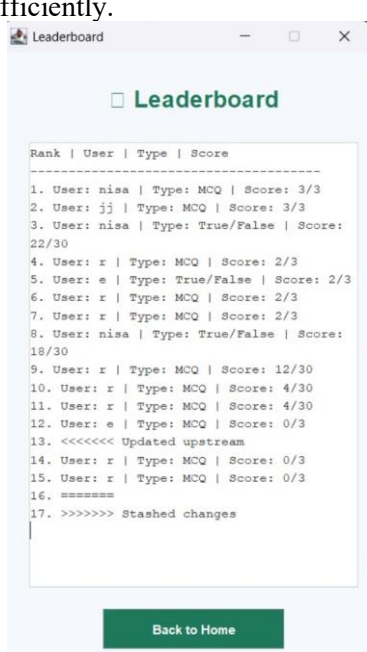
3.0 Testing and Debugging

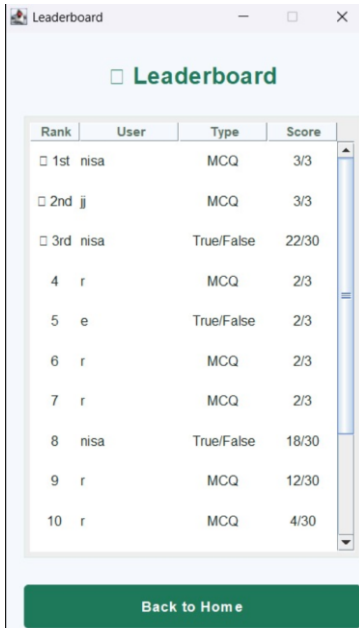
Class Name	LearningModule.java
Creator	Wan Adam Danish Bin Wan Abit (103014)
Tester	Qhairunnisa Shahirah Binti Zakaria (106089)
Test Scenario & Category	Scenario: Evaluating the readability, scaling, and layout of the educational poster images (10 pages) Category: UI Layout
Actions	1. Compile and run. 2. Click "Next" until reaching "SDG 4 Topic 10". 3. Observe the dimensions and clarity of the loaded image within the frame.
Expected Output	The poster should maximizing available space so that embedded text and poster is clear and visible to the user.
Actual Output	The poster image is small within its container, leaving excessive empty space and making the content difficult to read. 
Debugging & Resolution	Issue Found: The size was small and not suitable for the user readability. Fix Applied: Modified the image loading method in LearningModule.java. Implemented the right width, height for the image to better fit a larger preferred size before applying it to the JLabel. 

Class Name	Home.java
Creator	Qhairunnisa Shahirah Binti Zakaria (106089)
Tester	Wan Adam Danish Bin Wan Abit (103014)
Test Scenario	Scenario: Navigating from the Home dashboard to the Leaderboard and History pages Category: Runtime / Menu Navigation
Actions	1. Compile and launch Home.java 2. Click the “Leader” and “HistoryPage” buttons from the main menu grid.
Expected Output	The Home frame should close, and the respective LeaderBoard or HistoryPage module should launch and become visible.
Actual Output	The Home window disposed successfully, but the target module failed to open. The application immediately terminates when clicking Leaderboard and History.
Debugging and Resolution	Issue Found: A case sensitivity mismatch in the switch statement within the handleMenuClick() method. The menu items array defined the strings as “LeaderBoard” and “History”, but the switch cases were listening for “LeaderBoard” and “HistoryPage”  Fix Applied: Updated the switch cases on lines 497 and 503 to perfectly match the array values (changed to “Leaderboard” and “History”)

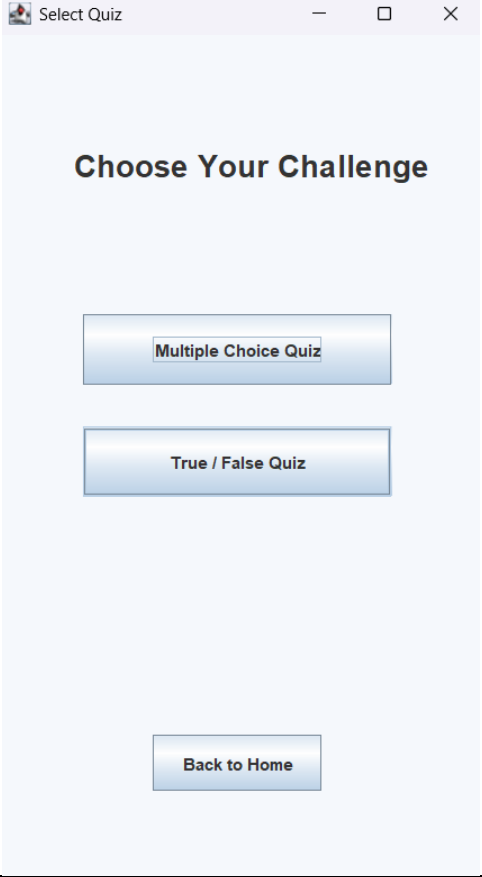
Class Name	MCQModule.java & TrueFalseModule.java
Creator	Abdul Rahim Bin Idris (102368)
Tester	Wan Adam Danish Bin Wan Abit (103014)
Test Scenario and Category	Scenario: Completing a quiz and transition to the result screen
Actions	1. Launch a quiz module and answer all questions. 2. Click “Next” on the final question to trigger the showResult() method
Expected Output	The quiz frame should dispose, data should save to QuizScores.txt, and unified QuizResultPage GUI should launch smoothly.
Actual Output	The application launched the QuizResultPage, but also excuted an older JOptionPane popup simultaneously. This caused overlapping UI elements and threw exceptions because the popup was trying to attach to a frame that had already been disposed.
Debugging and Resolution	Issue Found: The gray JOptionPane result dialog was accidentally left at the bottom of the showResult() methods during the integration of new custom Result Page. 

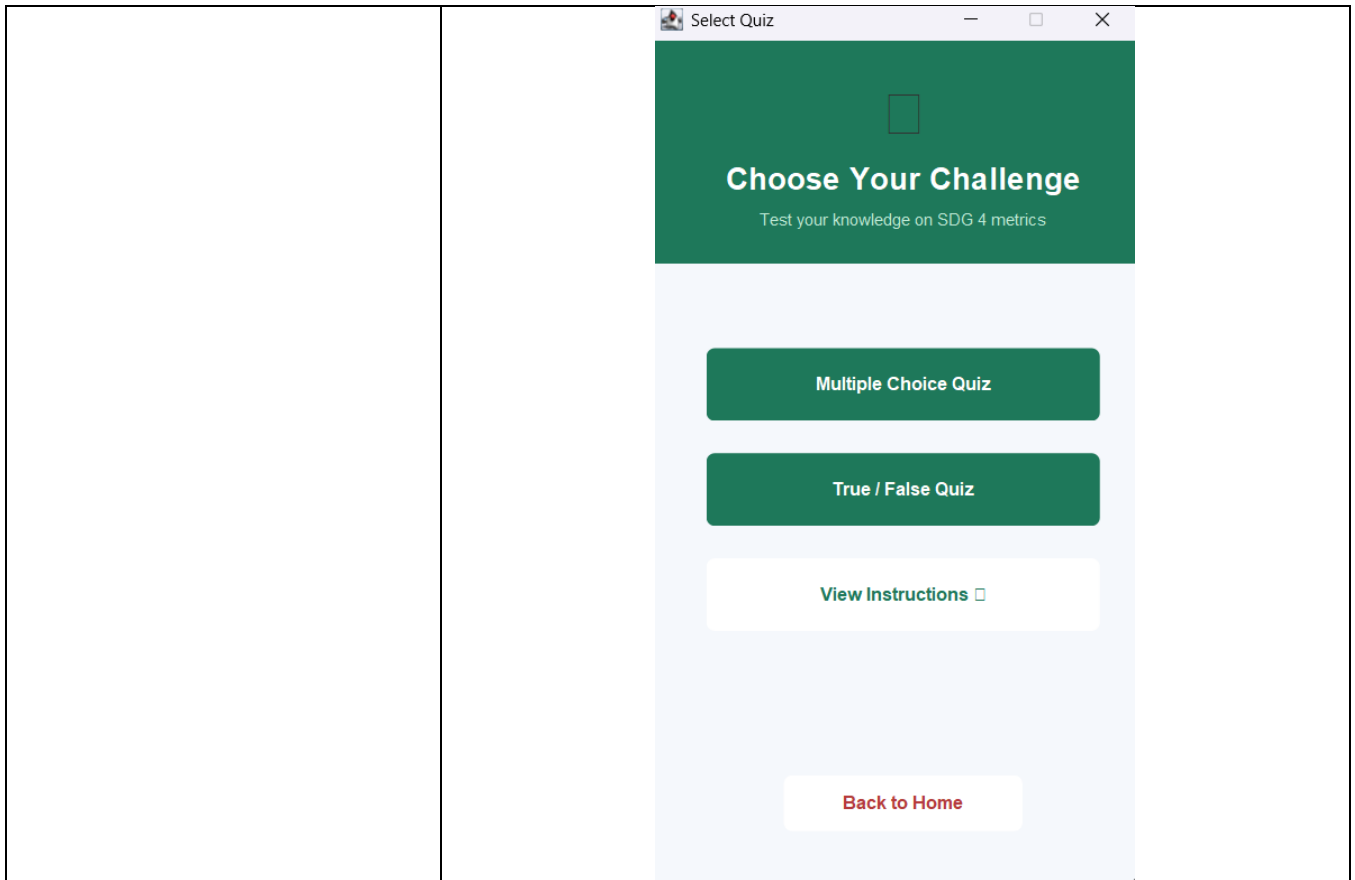
	 Fix Applied: Removed the redundant percentage calculations and JOptionPane.showMessageDialog() calls completely from lines 150-159 in MCQModule.java and lines 147-156 in TrueFalseModule.java
--	--

Class Name	LeaderBoard.java and HistoryPage.java
Creator	Putra Nuriman Akmal Bin Roziman (102917)
Tester	Abdul Rahim bin Idris (102368)
Test Scenario & Category	Scenario: Testing whether the leaderboard and quiz history pages can read quiz records from QuizScores.txt and display them correctly. Category: File Reading / GUI Display / Navigation
Actions	1. Compile all Java files using javac *.java. 2. Run the application using java Home. 3. Enter username and navigate to the Leaderboard and History pages. 4. Observe whether quiz scores are displayed correctly in the table format. 5. Click the “Back to Home” button to test navigation.
Expected Output	The leaderboard should display quiz rankings in a structured and user-friendly format, allowing users to easily compare their performance. Each record should clearly show the rank, username, quiz type, and score in separate columns.
Actual Output	The leaderboard initially displayed ranking information using a plain text area. Although the ranking data was correct, the layout appeared cluttered when the number of records increased, making it difficult for users to read and compare scores efficiently. 
Debugging & Resolution	Issue Found: The leaderboard used a JTextArea to display ranking records, causing poor readability and inconsistent alignment between columns. Fix Applied: The leaderboard interface was redesigned using JTable and DefaultTableModel. The ranking data was separated into individual columns (Rank, User, Type, and Score), improving readability and providing a cleaner user experience. The updated version was tested to ensure that score sorting, file reading, and navigation functions continued

	to work correctly after the UI modification. 
--	--

Class Name	MenuQuiz.java
Creator	Wan Adam Danish bin Wan Abit (103014) and Abdul Rahim bin Idris (102368)
Tester	Abdul Rahim bin Idris (102368)
Test Scenario & Category	Scenario: Testing the MenuQuiz to observe the buttons for MCQ and TrueFalse are functional or not.
Actions	1. Compile and run Home.java 2. Enter username and navigate to Quiz. 3. Click any of the quiz buttons to see if it moves to the respective page. 4. Click the “Back to Home” button to navigate back to Home screen.
Expected Output	The menu should be able to function well with both quiz buttons operating and navigate to the next page. The menu must also be able to return to the main page, Home with the “Back to Home” button being clicked.
Actual Output	The menu was able to navigate to the next page and also returned to Home screen. However the UI of the MenuQuiz did not look appealing to the users, which would entirely affect the whole user experiences.

	
Debugging & Resolution	Issue Found: The original “MenuQuiz” interface featured a basic, unstyled layout that lacked design cohesion with the main application theme. It relied on default boxy operating system buttons and lacked a visual anchor, making the navigation screen feel text-heavy and unpolished. Fix Applied: A premium corporate green header banner was integrated into the top of the frame to establish immediate visual alignment with the dashboard theme. The standard navigation blocks were replaced with custom anti-aliased, rounded buttons featuring dynamic mouse hover listeners that shift background states smoothly. Additionally, the <code>showInstructions()</code> was implemented with the help of <code>JOptionPane</code> to provide useful informations to users on what to expect before picking the quiz.



Class Name	Gamifiable.java
Creator	Wan Adam Danish Bin Wan Abit
Tester	Qhairunnisa Shahirah Binti Zakaria
Test Scenario & Category	Verify that the overloaded addPoints(int points, int bonus) method correctly calculates and adds the combined score.
Actions	1. Initialize test class object that implements the Gamifiable interface 2. Set initial score to 0 3. Call addPoints (50,10) on the object 4. Retrieved updated score
Expected Output	The system should update the object's score accordingly.
Actual Output	The system correctly calculated the sum and updated the object's score
Debugging & Resolution	No debugging needed. <pre> 1+ //Creator: Wan Adam Danish 2+ //Tester: Qhairunnisa Shahirah 3 //Description: Interface defining the required gamification methods. 4 public interface Gamifiable { 5 void addPoints(int points); 6 void addPoints(int points, int bonus); //Demonstrates Method Overloading 7 void awardBadge(int totalScore); 8 } 9 </pre>

4.0 Proof of Discussions

~ nicha +60 17-454 8330

unimas-my.sharepoint.com
https://unimas-my.sharepoint.com/:w:/g/personal/106089_siswa_unimas_my/IQCIQmT3WyOCTbGrDjnt4eHBAU3K3qv1kua_iIRYvE5YjM?e=yo8Was
unimas-my.sharepoint.com

Link Java:
Part A (Project Design):
https://unimas-my.sharepoint.com/:w:/g/personal/106089_siswa_unimas_my/IQCIQmT3WyOCTbGrDjnt4eHBAU3K3qv1kua_iIRYvE5YjM?e=yo8Was
Part B (Code+Documentation):
Code: https://github.com/Xevphln/G05_SE-G7-Project
Documentation: https://unimas-my.sharepoint.com/:w:/g/personal/106089_siswa_unimas_my/IQDEqrROGN5iSYMP0X7MPpPSAZUCQC4DsCxiQjpC4tQuZME?e=L49Z22

~ nicha pinned a message

Wednesday

Abdul Rahim/Xevphln
Group Leader

Salam yall, I've done my part for Track 2. Also I would like to point out that there is a slight error with the UML diagram where in AbstractQuizModule(timelimit is a method or attribute?), IQuiz (in the code itself shows boolean return value but in uml shows void and affecting both MCQModule and TFModule).

JP SE G07
ColdShadow, Nisa, Putra, You

Nisa: Link Java: Part A (Project Design): https://unimas-my.sharepoint.com/:w:/g/personal/106089_siswa_unimas_my/IQCIQmT3WyOCTbGrDjnt4eHBAU3K3qv1kua_iIRYvE5YjM?e=yo8Was

Today

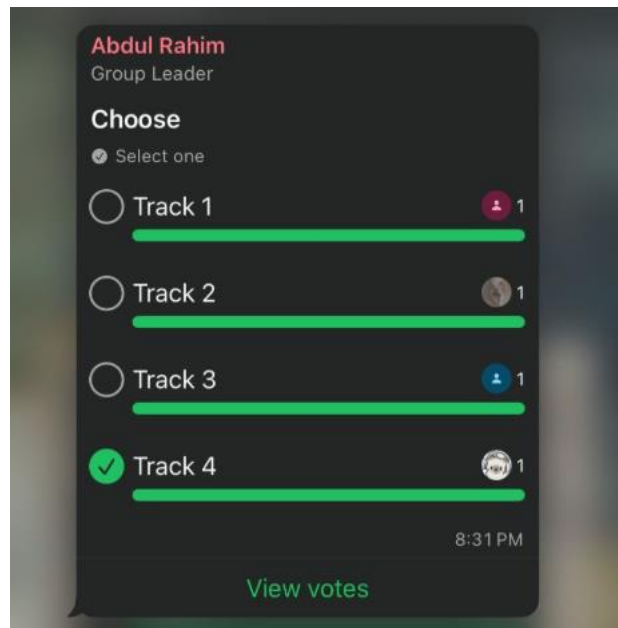
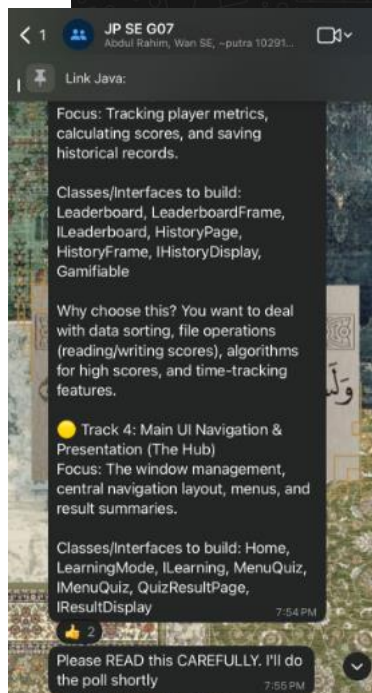
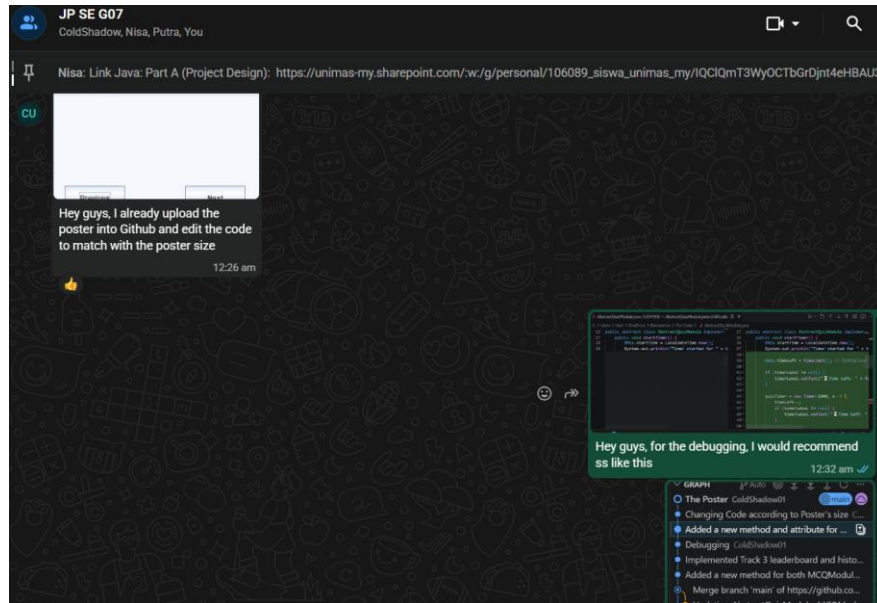
14. User: x | Type: MCQ | Score: 0/3
15. User: x | Type: MCQ | Score: 0/3
16. =====
17. >>>>>> Stashed changes

Bfr 8:10 am ✓

Rank	User	Type	Score
1st	nisa	MCQ	3/3
2nd	j	MCQ	3/3
3rd	nisa	True/False	22/30
4	r	MCQ	2/3
5	e	True/False	2/3
6	r	MCQ	2/3
7	r	MCQ	2/3
8	nisa	True/False	18/30
9	r	MCQ	12/30
10	r	MCQ	4/30

After 8:10 am ✓

@Putra Unimas Also I tested your code 8:10 am ✓




JP SE G07
 Abdul, +60 17-454 8330, +60 10-279 5150, You


 +60 17-454 8330: Link Java: Part A (Project Design): <https://unimas-my.sharepoint.com/:w:/g/person/...>

Abdul Rahim/Xevphin
 Group Leader



Okay, I've already done something with the MCQModule and TrueFalseModule. The UML needs to add a new attribute (- parentHome : Home) and a new method (+ setParent(h : Home) : void) for these modules.

01:19

~putra 102917 +60 10-279 5150



02:30

~putra 102917 +60 10-279 5150



02:31

Abdul Rahim/Xevphin
 Group Leader

@all Salam everyone, what slot should we take for the presentation?

14:26